

PROGRAMAÇÃO EM HASKELL



Capítulo 4 - Definindo Funções

Expressões condicionais

Como na maioria das linguagens de programação, funções podem ser definidas usando **expressões condicionais**

```
abs :: Int → Int  
abs n = if n ≥ 0 then n else -n
```

abs pega um inteiro **n** e retorna **n** se ele for não-negativo e **-n** caso contrário

Expressões condicionais podem ser aninhadas

```
signum :: Int → Int
signum n = if n < 0 then -1
           else if n == 0 then 0
                else 1
```

Nota

- Em Haskell, expressões condicionais devem **sempre** ter a cláusula **else**, o que evita possíveis problemas de ambiguidade com condicionais aninhados

Equações protegidas (*guarded*)

Como alternativa às expressões *if*, funções podem ser definidas usando *guarded equations*

```
abs n | n ≥ 0      = n  
      | otherwise = -n
```

Como anteriormente, mas usando
guarded equations

Equações protegidas podem ser usadas para facilitar a leitura de definições envolvendo várias condições

```
signum n | n < 0      = -1  
         | n == 0     = 0  
         | otherwise = 1
```

Nota

- A condição em qualquer outro caso **otherwise** é definida no prelúdio por **otherwise = True**

Casamento de padrões

Muitas funções têm uma definição particularmente clara usando casamento de padrões (*pattern matching*) em seus argumentos

```
not :: Bool → Bool  
not False = True  
not True  = False
```

not mapeia **False** para **True**
e **True** para **False**

Funções podem ser definidas de muitas maneiras diferentes usando a **casamento de padrões**, por exemplo

```
(&&) :: Bool → Bool → Bool
True  && True   = True
True  && False  = False
False && True   = False
False && False  = False
```

pode ser definida de forma **mais compacta** por

```
True && True = True
_    && _    = False
```

Entretanto, a definição a seguir é **mais eficiente**, pois evita avaliar o segundo argumento se o primeiro argumento for Falso:

```
True    && b = b  
False  && _ = False
```

Nota

- O símbolo de sublinhado é um padrão curinga (*wildcard*) que corresponde a qualquer valor de argumento

- Padrões são casados em ordem, por exemplo, a definição a seguir sempre retorna **False**

```
_      && _      = False
True && True = True
```

- Padrões podem não repetir variáveis, por exemplo, a seguinte definição resulta em erro

```
b && b = b
_ && _ = False
```

Padrões de listas

Internamente, cada lista não vazia é construída pelo **uso repetido de um operador** (`:`) chamado "cons", que adiciona um elemento ao início de uma lista

[1, 2, 3, 4]

Significa **1 : (2 : (3 : (4 : [])))** ou apenas **1 : 2 : 3 : 4 : []**

Funções em listas podem ser definidas usando o padrão **x:xs**

```
head :: [a] → a  
head (x:_) = x
```

```
tail :: [a] → [a]  
tail (_:xs) = xs
```

head e **tail** mapeiam qualquer lista não vazia para a cabeça e a cauda da lista, respectivamente

Nota

- O padrão $x:xs$ corresponde apenas a listas não vazias

```
> head []  
*** Exception: empty list
```

- O padrão $x:xs$ deve ser colocado entre parênteses porque a aplicação da função tem prioridade sobre $(:)$, por exemplo, a seguinte definição resulta em erro

```
head x:_ = x
```

Expressões lambda

Funções podem ser construídas sem serem nomeadas usando expressões lambda

$$\lambda x \rightarrow x + x$$

Função anônima que pega um número x e retorna o resultado $x + x$

Nota

- O símbolo λ é a letra grega **lambda** e é digitado no teclado como uma barra invertida `\`
- Em matemática, funções anônimas são em geral denotadas usando o símbolo $\mapsto$, como em $x \mapsto x + x$
- Em Haskell, o uso do símbolo λ para funções anônimas vem do lambda calculus, a teoria das funções na qual Haskell se baseia

Por que o lambda é útil?

Expressões lambda podem ser usadas para dar um significado formal às funções definidas usando *currying*

Por exemplo

```
add :: Int → Int → Int  
add x y = x + y
```

significa

```
add :: Int → (Int → Int)  
add = λx → (λy → x + y)
```

Expressões lambda podem ser usadas para evitar nomear funções que são referenciadas **apenas uma vez**

Por exemplo

```
odds n = map f [0..n-1]
      where
        f x = x*2 + 1
```

pode ser simplificada para

```
odds n = map ( $\lambda x \rightarrow x*2 + 1$ ) [0..n-1]
```


Seções de operador

Um operador escrito **entre** seus dois argumentos pode ser convertido em uma **função curried** escrita antes de seus dois argumentos, usando parênteses

Por exemplo

```
> 1+2
```

```
3
```

```
> (+) 1 2
```

```
3
```

Essa convenção permite que um dos argumentos do operador seja incluído entre parênteses

Por exemplo

```
> (1+) 2  
3  
  
> (+2) 1  
3
```

Em geral, se $\oplus$ é um operador, então as funções da forma $(\oplus)$, $(x\oplus)$ e $(\oplus y)$ são chamadas de **seções**

Por que seções são úteis?

Funções úteis às vezes podem ser construídas de maneira simples usando seções, por exemplo

$(1+)$ - função sucessora

$(1/)$ - função de reciprocidade

$(*2)$ - função de duplicação

$(/2)$ - função de redução pela metade

Exercícios

(1) Use funções de biblioteca para definir a função

```
halve :: [a] -> ([a],[a])
```

que separa uma lista de tamanho par em duas metades, por exemplo

```
> halve [1,2,3,4,5,6]  
([1,2,3],[4,5,6])
```

Exercícios

(2) Defina a função

```
third :: [a] -> a
```

que retorna o **terceiro elemento** de uma lista que contenha ao menos três elementos, usando

(a) **head** e **tail**

(b) indexação de listas **!!**

(c) casamento de padrão

Exercícios

- (3) Considere uma função `safetail` que se comporta como a função `tail`, exceto que mapeia a lista vazia para a lista vazia, enquanto `tail` dá um erro neste caso

Defina `safetail` usando

- (a) Expressão condicional
- (b) Guarded equations
- (c) Casamento de padrões

Dica: a função de biblioteca `null :: [a] → Bool` testa se uma lista está vazia

- (4) Forneça ao menos três definições possíveis para o operador lógico ou (||) usando casamento de padrão
- (5) Redefina a seguinte versão de (&&) usando condicionais em vez de padrões

```
True && True = True  
_      && _   = False
```

- (6) Faça o mesmo para a seguinte versão

```
True  && b = b  
False && _ = False
```

(7) Mostre como o significado da definição da seguinte função *curried* pode ser formalizada em termos de expressões lambda

```
mult :: Int -> Int -> Int -> Int  
mult x y z = x*y*z
```


(8) O **algoritmo de Luhn** é usado para verificar os números dos cartões bancários em busca de erros simples, como digitar incorretamente um dígito, e procede da seguinte forma

- considere cada dígito como um número separado
- da direita para a esquerda, a partir do penúltimo dígito, dobre alternadamente cada dígito até o primeiro
- subtraia 9 de cada um dos números dobrados que agora é maior que 9
- some todos os números resultantes, dobrados e não dobrados
- se o total é divisível por 10, o número é válido

Defina uma função

```
1uhnDouble :: Int -> Int
```

que dobra um dígito e subtrai 9 se o resultado for maior que 9, por exemplo

```
> 1uhnDouble 3  
6
```

```
> 1uhnDouble 6  
3
```

Usando a função `luhnDouble` e a função `mod`, que calcula o resto de uma divisão inteira, defina a função

```
luhn :: Int -> Int -> Int -> Int -> Bool
```

que decide se um número de cartão bancário de quatro dígitos é válido, por exemplo

```
> luhn 1 7 8 4  
True
```

```
> luhn 4 7 8 3  
False
```